Sudoku AI as a Logic-Based Agent

Predicate Calculus Style Encoding + Z3

Student Project

Artificial Intelligence / Knowledge Representation

October 16, 2026

Outline

1. Problem and goal
2. Predicate-calculus foundation
3. Sudoku knowledge base
4. Z3 implementation
5. Results, limitations, and demo

Problem statement

Given a partially completed 9×9 Sudoku:

- Fill every cell with a digit from 1 to 9.
- Each digit occurs once in every row.
- Each digit occurs once in every column.
- Each digit occurs once in every 3×3 box.
- Preserve all original clues.

Input puzzle

Constraints

$\Downarrow$

Z3 solver

$\Downarrow$

Completed grid

Predicate-calculus foundation

Define the predicate:

$$\mathrm{Val}(r, c, d)$$

meaning cell (r, c) contains digit d.

- Objects: row, column, and digit indices.
- Predicate: $\mathrm{Val}(r, c, d)$.
- Quantifier: $\forall$ (for all applicable rows, columns, and cells).
- Logical connectives: $\wedge$ and $\Rightarrow$.

Example cell rule:

$$\forall r, c \in \{0, \ldots, 8\}, \quad \sum_{d=1}^{9} \mathbf{1}[\mathrm{Val}(r, c, d)] = 1$$

Here $\mathbf{1}[P]$ is 1 when P is true and 0 otherwise.

Knowledge-base constraints

For every cell:

$$\text{exactly one digit } d \in \{1, \ldots, 9\}$$

For every row r and digit d:

$$\sum_{c=0}^{8} \mathbf{1}[\text{Val}(r, c, d)] = 1$$

For every column c and digit d:

$$\sum_{r=0}^{8} \mathbf{1}[\text{Val}(r, c, d)] = 1$$

For every 3×3 box and digit d, exactly one cell in that box has digit d.

Z3: defining the predicate

```
from z3 import Function, IntSort, BoolSort

Val = Function(
    "Val",
    IntSort(), IntSort(), IntSort(),
    BoolSort()
)
```

- `Val(r,c,d)` is a Boolean-valued predicate.
- It is true when cell (r, c) contains digit d.
- The finite Sudoku domain is rows/columns 0–8 and digits 1–9.

Z3: quantified cell constraint

```
cell_digit_count = Sum([
    If(Val(r, c, digit), 1, 0)
    for digit in range(1, 10)
])

solver.add(ForAll(
    [r, c],
    Implies(
        And(r >= 0, r < 9, c >= 0, c < 9),
        cell_digit_count == 1
    )
))
```

- `ForAll`: universally quantified rule.
- `If`: turns each Boolean predicate into 1 or 0.
- `Sum(...)=1`: exactly one digit is true for the cell.

Rows, columns, boxes, and clues

```
# Example: each digit appears once in each row
row_count = Sum([
    If(Val(r, column, d), 1, 0)
    for column in range(9)
])
solver.add(ForAll(
    [r, d],
    Implies(
        And(r >= 0, r < 9, d >= 1, d <= 9),
        row_count == 1
    )
))

# A given clue becomes a fact
solver.add(Val(row, column, given))
```

The same exact-one pattern is used for columns; each 3×3 box is constrained explicitly.

Inference and execution

Puzzle clues $\rightarrow$ Knowledge base $\rightarrow$ Z3 $\rightarrow$ Model

1. Add the Sudoku rules and given clues.
2. Call `solver.check()`.
3. If the constraints are satisfiable, read the model.
4. Convert true $\mathrm{Val}(r, c, d)$ atoms into the 9×9 solution.

Interpretation: the solver finds an assignment satisfying all encoded rules; it does not guess numbers by visual pattern matching.

Interactive Colab interface

The notebook interface provides:

- Click an empty cell and select a digit using the number pad.
- Check row, column, and box conflicts.
- Request a hint.
- Ask Z3 to solve the puzzle.
- Reset to the original clues.

The interface runs inside Google Colab using Python widgets—no local host or separate web server.

Testing and limitations

Tests

- Each row contains digits 1–9 exactly once.
- Each column contains digits 1–9 exactly once.
- Each 3×3 box contains digits 1–9 exactly once.
- Original clues remain unchanged.

Limitations

- The demo uses a built-in puzzle.
- Quantified SMT solving can be slower than a non-quantified integer model.
- The model is a predicate-calculus-style quantified constraint encoding in Z3, not a standalone theorem prover.

Summary

- Model: $\mathrm{Val}(r, c, d)$.
- Foundation: predicate-calculus-style rules with universal quantification.
- Solver: Z3 / SMT satisfiability.
- Output: a Sudoku grid satisfying every clue and rule.

Thank you!